

Collections Framework

1. What is a collection in programming?
2. Can you give an example of a collection?
3. What is a Collection in Java?
4. What was the limitation of Collections before Java
5. What was the main purpose of introducing the Collection Framework in Java 1.2?
6. How did the introduction of interfaces in Java improve the use of generic methods?
7. What is the Collection interface in Java?
8. What are the main types of collections in the Java Collections Framework?
9. What is the FIFO principle in the context of collections?
10. What is the difference between a List and a Set in Java Collections?
11. What is a List in Java Collections Framework?
12. What are some classes that implement the List interface in Java?
13. What is the purpose of the Iterable interface in Java?
14. How do Sets relate to the Iterable interface in Java?
15. What is the role of the Map interface in the Java Collections Framework?
16. Can you name some implementations of the Map interface in Java?
17. What is the role of the root interface in the Java Collections Framework?
18. Can interfaces in Java be instantiated directly?
19. What is the purpose of the Collection interface in Java?
20. How does the List interface relate to the Collection interface in Java?
21. What is the primary use case for the List interface in Java?
22. Name two implementations of the List interface in Java.
23. What is a key characteristic of the ArrayList in Java?
24. Can ArrayLists in Java contain duplicate elements?
25. What is the main advantage of using an ArrayList over an array in Java?
26. How does an ArrayList manage its size when it becomes full?
27. What is an ArrayList in Java?
28. What does it mean to use a parameterized type with ArrayLists?
29. What is the purpose of using generics in Java when working with collections like ArrayList?
30. How does zero-based indexing work in an ArrayList?
31. In an ArrayList, the first element is accessed with index 0, the second with index 1, and so on.
32. What is the method to get the size of an ArrayList in Java?
33. How can you iterate over an ArrayList in Java?
34. What does the 'contains' method do in a LinkedList?
35. How do you print elements from a LinkedList in Java?
36. What method is used to remove an item from a list in Java?
37. What happens to the size of a list after an item is removed?
38. What method can be used to insert an element between two existing elements in a list in Java?
39. How does the 'set' method affect the elements in a Java list?
40. What is the primary method used to convert a Vector to a String in Java?
41. How does synchronization in Vector affect its performance compared to ArrayList?
42. What is an Abstract List in Java?
43. How does an ArrayList differ from a regular array in Java?
44. What is the main difference between an array and an array list in Java?
45. What happens internally when the number of elements in an array list exceeds its current capacity?
46. What is the initial capacity of an ArrayList in Java?

47. What is the difference between capacity and size in an ArrayList?
48. What is the default capacity of an ArrayList in Java?
49. What happens when an element is added to an ArrayList in Java?
50. What happens when an internal array in a list is full?
51. What is the default initial capacity of a list in Java?
52. What is the growth factor for an internal array in Java when it needs to resize?
53. What happens to the elements of the old array when a new larger array is created in Java?
54. What happens to the elements in an array when one is removed?
55. In the context of a HashMap, why is it important to understand element removal?
56. What happens to the elements in a list when an element is removed?
57. The remaining elements are shifted one position to the left to fill the gap.
58. Why should the size of a list be adjusted after removing an element?
59. What does 'initial capacity' mean in the context of a data structure like a List?
60. How does the initial capacity affect the performance of a List in Java?
61. What happens to the capacity of a list when it exceeds its initial capacity in Java?
62. What is the significance of capacity in a HashMap?
63. How can reflection be used to check the capacity of a HashMap?
64. What is the purpose of using reflection in Java when dealing with private fields?
65. How does the capacity of a data structure like a LinkedHashMap change when elements are added?
66. What method is used to adjust the size of a list in Java after removing elements?
67. What happens to the internal size of a list when elements are removed but not trimmed?
68. What happens to the internal size of a list when elements are removed but not trimmed?
69. What are the main characteristics of an Identity HashMap?
70. What is the significance of the 'ArrayList' in Java?
71. What does the 'Arrays.asList()' method return in Java?
72. What is a Fixed Size List in Java?
73. What is the difference between a Fixed Size List and a modifiable List in Java?
74. What is the method to create a new list based on an existing list in Java?
75. What happens when you modify a new list created from an existing list in Java?
76. How can elements be added to a list in Java?
77. What is the difference between adding an element at the end of a list and at a specific index?
78. What does the 'add' method do in Java Lists?
79. What is the purpose of the 'addAll' method in Java Lists?
80. What happens when you use the remove method on a list in Java?
81. What will happen if you try to print a list after removing an element?
82. What is the significance of using the correct method when removing elements from a collection in Java?
83. What happens when you call the remove method with an index in Java collections?
84. What method can be used to convert a List to an Array in Java?
85. What does the 'size()' method do in a List?
86. What is the purpose of passing a zero size when creating a new array in Java?
87. How can you sort a list in Java using the Collections class?
88. What is the time complexity for accessing an element in a list using its index?
89. What is the worst-case time complexity for adding or removing an element from a list?
90. What is the time complexity of looping through an entire list in Java?
91. What is a Comparator in Java?
92. What is the purpose of the Comparator interface in Java?
93. How can you sort a list of words by their length using a Comparator?

94. What is the purpose of a custom comparator in Java?
95. How do you implement a custom comparator using a lambda expression in Java?
96. What is a linked list in data structures?
97. How does a linked list differ from an array in terms of memory allocation?
98. What is a circular linked list?
99. What are the key differences between a singly linked list and a doubly linked list?
100. What is the time complexity for adding an element at the beginning of a linked list?
101. How does the 'remove' method work in a linked list?
102. What is a key feature of a Vector in Java?
103. When should you use a Vector instead of an ArrayList in Java?
104. What is the initial capacity of a Vector in Java when created without specifying a size?
105. How does the capacity of a Vector change when elements are added beyond its current size?
106. What does the term 'Last In First Out' (LIFO) mean in the context of a stack?
107. What are the two main operations associated with a stack in Java?
108. What is the purpose of the 'pop' method in a stack?
109. How can you view the top element of a stack without removing it?
110. What is a HashMap in Java?
111. What are the key characteristics of a HashMap?
112. What is the time complexity of basic operations in a HashMap?
113. What happens in a HashMap when two different keys produce the same hash code?
114. What is collision handling in the context of hash maps?
115. What happens to a linked list in a hash map when the number of entries exceeds a certain threshold?
116. What is rehashing in the context of hash maps?
117. When does rehashing typically occur in a hash map?
118. What is the purpose of an LRU Cache?
119. What is garbage collection in Java?
120. What is a strong reference in Java?
121. What is a weak reference in Java?
122. When is a weak reference automatically removed in Java?
123. What is the primary difference between a regular HashMap and an IdentityHashMap in Java?
124. How does the insertion process in an IdentityHashMap handle key collisions?
125. How does Identity HashMap handle hash codes for objects?
126. What is the purpose of a hash code in a hash map?
127. When would you use a comparator in Java collections?
128. What is required when using the sort method on a list in Java?
129. What happens if you provide a null comparator to the sort method?
130. What does passing 'null' as a comparator to the sort method in Java indicate?
131. When should a custom comparator be provided in Java sorting?
132. What is the purpose of a Comparator in Java?